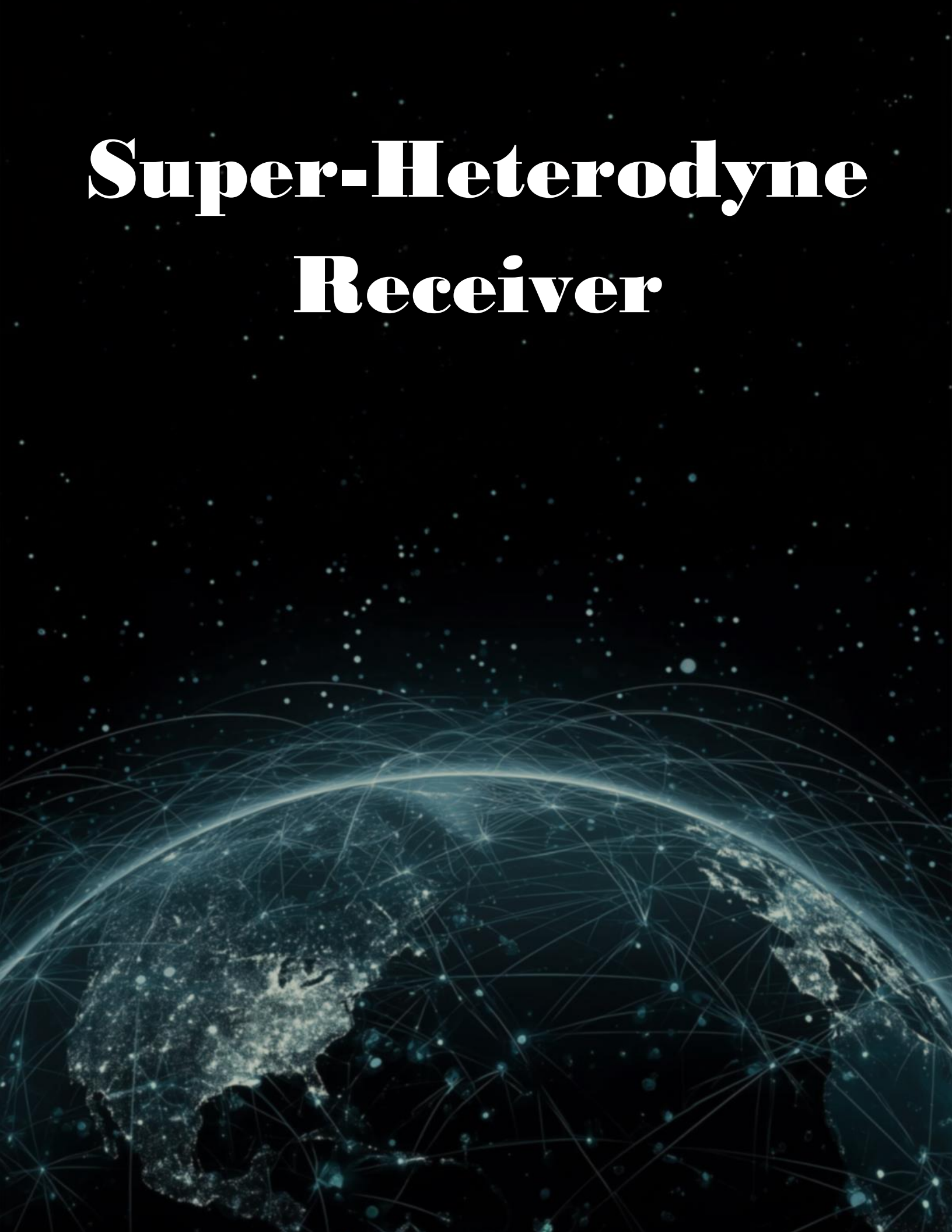
Super-Heterodyne Receiver



Cairo University
Faculty of Engineering
Department of Electronics and Electrical Communications

Presented to:

Dr. Ahmed Hesham
Eng. Mohamed Khaled

Presented by Team 22:

Name	ID
Sief Khaled Refaat	91240376
Sief Osama Elsayed	91240367

Table of Contents

1. Introduction……..……..….4

2. System Architecture…….…….……...4

3. Project Software Architecture……..……...5

4. Data Handling and Signal Preparation……..……..……6

4.1. System Configuration……..…….6

4.2. Audio Loading …….……..6

4.3. Signal Interpolation ……..……..……....7

4.3.1. The Motivation……...…..7

4.3.2. Analysis……...….7

4.3.3. Applied Interpolation……...……9

5. Transmitter Design ……..9

5.1. DSB Modulation …….……9

5.2. Frequency Division Multiplexing……..…….....10

6. Receiver Design …….....11

6.1. RF Stage …….....11

6.2. Mixer Stage ……...…11

6.3. IF Stage……..12

6.4. Baseband Mixer……13

6.5. Baseband LPF……13

6.6. Baseband Decimation……14

7. Super Heterodyne Receiver ……...……..…14

8. Verification and Testing……..……...……...15

8.1. Verification Plan……15

8.2. Verification Tests……16

8.2.1. Test Signal Loading…….……..16

8.2.2. Test Interpolation……...16

8.2.3. Test Modulation……..……18

8.2.4. Test RF Stage…….……19

8.2.5. Test Mixer Stage…….……...20

8.2.6. Test IF Stage……..…….21

8.2.7. Test Baseband Mixer Stage…….……...21

8.2.8. Test Baseband LPF…….....22

8.2.9. Test Decimation……..……22

8.2.10. Test Signal Recovery…….……..23

8.3. Automated Test File……24

9. Main System ……..…….……..……...25

10. Experiments and Results……...…26

10.1. Removing RF Bandpass Filter……...26

10.2. Local Oscillator Mismatch……27

11. Version Control and Project Organization……..…...30

12. Conclusion…….....31

Super-heterodyne Receiver

1. Introduction

Analog communication systems are widely used for transmitting information over long distances using modulation techniques that allow signals to be efficiently transmitted through a shared medium. One of the most widely used receiver structures in radio communication is the **superheterodyne receiver**, which improves signal selectivity and sensitivity by converting the received signal to an intermediate frequency before demodulation.

In this project, a complete analog communication system is simulated using **MATLAB**. Several audio signals are used as message sources and are modulated using **double sideband suppressed carrier (DSB-SC)** modulation. The modulated signals are then combined using **frequency division multiplexing (FDM)** to simulate multiple radio stations transmitting simultaneously at different carrier frequencies.

The objective of this work is to design and implement the main functional blocks of the transmitter and receiver, analyze the behavior of the system through spectral visualization, and evaluate the quality of the recovered audio signals. The system performance is assessed by comparing the transmitted and recovered signals using signal analysis and signal-to-noise ratio measurements.

2. System Architecture

The system begins with the **input audio message**, which is modulated using **AM modulation** to shift the signal from baseband to a radio frequency carrier. Multiple modulated signals are combined using **frequency division multiplexing (FDM)** so that several stations occupy different frequency bands in the transmitted spectrum.

At the receiver, the signal first passes through the **RF stage**, where a band-pass filter selects the desired station and suppresses neighboring channels.

The filtered signal is then applied to a mixer, where it is multiplied by a **local oscillator** to translate the selected station to an **intermediate frequency (IF).**

In the next stage, an **IF filter** isolates the desired IF component and removes unwanted mixing products. Finally, the signal is translated to **baseband** through another mixing operation, and a **low-pass filter** recovers the original audio message at the receiver output.

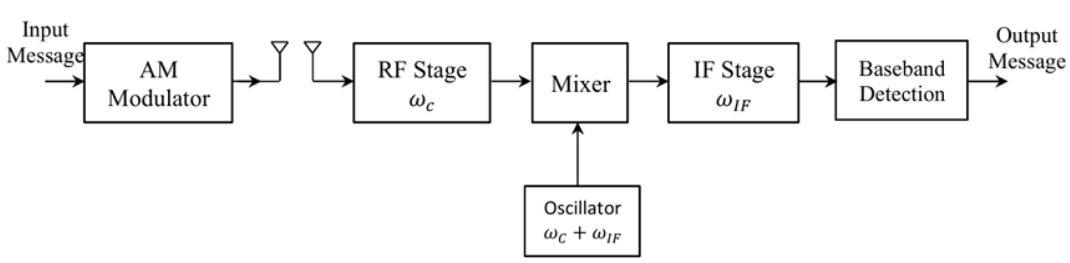


Figure 1: Super Heterodyne Receiver System Architecture

3. Project Software Architecture

The MATLAB implementation was organized as a modular software project rather than a single script. The system was divided into **multiple functional components** including signal processing **modules**, testing **utilities**, **configuration** files, and **data storage** directories. This structure improves code readability, simplifies debugging, and allows independent verification of each subsystem before integrating the complete receiver chain.

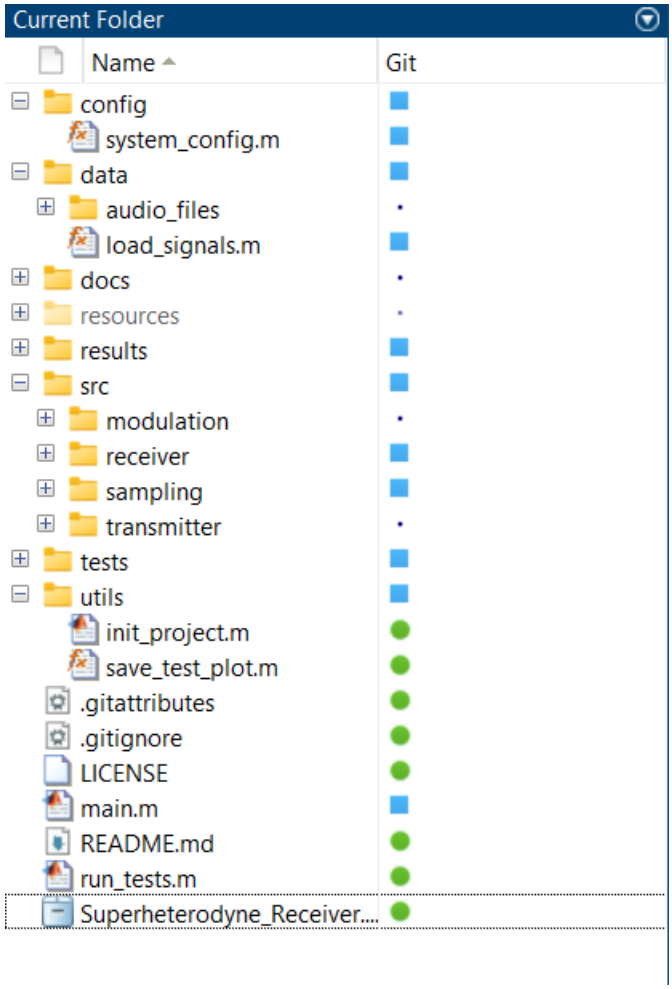


Figure 2: Project Software Structure

- **Config**: contain all system configurable parameters
- **Src** : Contains the main signal processing modules implementing transmitter and receiver stages
- **Tests** : contains verification scripts used to test individual components of the system
- **Utils** : contains helper utilities such as project initializer and automated tests execution
- **Data**: stores provided audio station radio signals
- **Results**: stores all simulation plots and receiver output
- **Run_tests**: runs all verification tests sequentially
- **Main**: Execute the complete Super-heterodyne Receiver system

This organization separates system logic from testing and configuration components, which follows common software engineering practices used in signal processing projects.

4. Data Handling and Signal Preparation

4.1. System Configuration

A configuration function is used to define the main parameters of the communication system in one place. These parameters include the **base carrier frequency**, **channel spacing**, **intermediate frequency**, **signal bandwidth**, **interpolation factor**.

The configuration also includes a **station selection parameter**, which allows the receiver to tune to the desired channel.

```
function config = system_config()
% Carrier base frequency
config.Fc0 = 100e3;
% Channel spacing
config.deltaF = 30e3;
% Intermediate frequency
config.IF = 15e3;
% Band Width
config.BW = 6e3;
% Selected station index
% 1 -> 100 kHz ..... % 2 -> 130 kHz .....
config.station = 1;
% Interpolation factor
config.L = 60;
end
```

We are going to explain the criteria of the selection of each parameter in detail.

4.2. Audio Loading

Audio signals are loaded from the project audio directory using MATLAB's **audioread** function. The program automatically locates the project root directory and searches the folder **data/audio_files** for all **.wav** files.

Each audio file is read and stored in a cell array. Since the provided recordings are **stereo** signals, they are converted to **mono** by **averaging** the two channels. The processed signals are then stored in a **structure** containing the audio messages and their sampling frequency.

This structure is used later in the transmitter to access the input messages and maintain a consistent sampling rate across all signals.

```
function signals = load_signals()
% Locate project root directory (parent of source folder)
this_file = mfilename('fullpath');
[src_folder,~,~] = fileparts(this_file);
project_root = fileparts(src_folder);
% Define path to audio signals directory
audio_folder = fullfile(project_root,"data","audio_files");
% Retrieve all WAV files from the directory
files = dir(fullfile(audio_folder,"*.wav"));
```

```
  % Stop execution if no audio files are found
  if isempty(files)
    error("No .wav files found in %s",audio_folder)
  end
  % Determine number of input signals
  num_signals = length(files);
  % Preallocate cell array to store audio messages
  messages = cell(1,num_signals);
  % Read each audio file and process it
  for k = 1:num_signals
    % Construct full file path
    filepath = fullfile(files(k).folder,files(k).name);
    % Read audio signal and sampling frequency
    [audio,Fs] = audioread(filepath);
    % Convert stereo signals to mono by averaging channels
    if size(audio,2) == 2
        audio = mean(audio,2);
    end
    % Store processed signal
    messages{k} = audio;
end
% Store outputs in structured format
signals.messages = messages;
signals.Fs = Fs;
end
```

4.3. Signal Interpolation

4.3.1. The Motivation

The original audio signals have a sampling frequency of approximately **44.1 kHz**, while the carrier frequency used for transmission begins at **100 kHz**, with adjacent carriers spaced by **30 kHz**. When multiple signals are modulated, the highest carrier frequency can reach approximately **220 kHz** for five stations.

In addition to the carrier frequency, each signal occupies an audio bandwidth of roughly **5–6 kHz**. As a result, the required spectrum extends well beyond half of the original sampling frequency, which violates the **Nyquist sampling criterion** and leads to **aliasing** in the simulated RF signal.

To overcome this limitation, the sampling frequency must be increased before performing the modulation process. Instead of simple upsampling by inserting zeros, the signals are **interpolated** to generate additional samples and effectively increase the sampling rate.

This interpolation step produces a higher-resolution representation of the signal, allowing accurate modulation at the required RF carrier frequencies without introducing aliasing.

4.3.2. Analysis

Before determining the interpolation factor, it is necessary to estimate the **bandwidth (BW)** of the audio signals. This estimation is also required for the design of filters and other receiver stages. The bandwidth can be approximated by analyzing the signal spectrum and determining the frequency range that contains **99% of the total signal energy**.

By sweeping across the spectrum and identifying this limit, a practical estimate of the signal bandwidth can be obtained.

```
num_signals = length(messages);
for k = 1:num_signals
y = messages{k};
N = length(y);
Y = fftshift(abs(fft(y)).^2);
f = (-N/2:N/2-1)*(Fs/N);
% keep positive frequencies only
Y_pos = Y(N/2:end);
f_pos = f(N/2:end);
% cumulative energy
cum_energy = cumsum(Y_pos);
total_energy = cum_energy(end);
% find frequency containing 99% of energy
idx = find(cum_energy >= 0.99*total_energy,1);
BW = f_pos(idx);
fprintf("Estimated Audio BW of station %d ? %.2f Hz\n",k,BW);
end
```

```
>> test_signal_loading
Estimated Audio BW of station 1 ≈ 5793.49 Hz
Estimated Audio BW of station 2 ≈ 6934.05 Hz
Estimated Audio BW of station 3 ≈ 11576.31 Hz
Estimated Audio BW of station 4 ≈ 6575.40 Hz
Estimated Audio BW of station 5 ≈ 7174.15 Hz
```

Figure 3: Bandwidth Estimation of All Signals

The estimated bandwidth of the audio signals ranges between approximately **5 kHz** and **7 kHz** for most stations. One signal exhibited a higher bandwidth due to additional high-frequency components. For system design and filter implementation, a bandwidth of **6 kHz** was selected as a practical approximation for the audio signals.

After estimating the signal bandwidth, we can now analyze for the interpolation factor

$$F_s = 44.1\ KHz$$

$$BW = 6\ KHz$$

The maximum frequency can be reached in FDM is $f_m = f_c + (n_{max} - 1) \times 30 + BW$

$$f_m = 226\ KHz$$

By applying **Nyquist sampling criterion**

$$\boldsymbol{F_s > 2f_m}$$

$$F_s > 452\ KHz$$

Thus we can estimate the interpolation factor

$$L > {F_{s_{new}}}/{F_{s_{old}}}$$

$$\boldsymbol{L > 10.25}$$

The minimum interpolation factor required to satisfy the **Nyquist** criterion was calculated to be approximately **10.25**. Although smaller values such as 20 or 30 would theoretically be sufficient, increasing the interpolation factor improves **RF signal representation**, **spectral analysis**, and **digital filter implementation** in the simulation. However, very large interpolation factors increase **memory usage** and **computational complexity**. Therefore, an interpolation factor of **60** was selected as a practical compromise that provides **a large safety margin** above the minimum requirement while maintaining **reasonable computational efficiency**.

4.3.3. Applied Interpolation

```
function signals_out = interpolate_signal(signals_in, L)
% Extract input signals and original sampling frequency
messages = signals_in.messages;
Fs = signals_in.Fs;
% Determine number of signals
num_signals = length(messages);
% Preallocate cell array for interpolated signals
messages_interp = cell(1,num_signals);
% Process each signal individually
for k = 1:num_signals
% Retrieve current signal
signal = messages{k};
% Define original discrete-time index
old_time = (0:length(signal)-1)';
% Define new time index with higher resolution
new_time = (0:1/L:length(signal)-1)';
% Perform linear interpolation
signal_interp = interp1(old_time,signal,new_time,'linear');
% Store interpolated signal
messages_interp{k} = signal_interp;
end
% Store outputs in structured format
signals_out.messages = messages_interp;
% Update sampling frequency after interpolation
signals_out.Fs = L * Fs;
end
```

5. Transmitter Design

5.1. DSB Modulation

Each audio message signal is modulated using **DSB-SC (Double Sideband Suppressed Carrier)** modulation. In this process, a carrier signal with frequency $\boldsymbol{F_c}$ is generated for each station using the system sampling frequency.

The baseband message signal is then multiplied by the carrier signal to shift its spectrum from baseband to the corresponding radio frequency. This operation produces the modulated RF signal centered around the selected carrier frequency, which is later used in the **FDM multiplexing stage** of the transmitter.

The modulation process implemented in MATLAB is shown in the following function:

```
function modulated_signal = dsb_sc_modulate(message, Fc, Fs)
 % Determine number of samples
 N = length(message);
 % Time axis
 t = (0:N-1)' / Fs;
 % Generate carrier
 carrier = cos(2*pi*Fc*t);
 % Perform DSB-SC modulation
 modulated_signal = message .* carrier;
end
```

5.2. Frequency Division Multiplexing

After modulation, the signals of all stations are combined to form a **Frequency Division Multiplexed (FDM)** signal. Each audio message is first modulated using DSB-SC with a different carrier frequency. The first station is assigned a carrier at **100 kHz**, and the carrier frequency of each subsequent station is increased by **30 kHz**.

Since the audio files may have different lengths, all signals are truncated to the length of the shortest signal to ensure consistent processing. Each truncated message is then modulated with its corresponding carrier, and all modulated signals are summed together to generate the final **multiplexed FDM signal**, which contains all stations occupying separate frequency bands in the RF spectrum.

The FDM process implemented in MATLAB is shown in the following function:

```
function fdm_signal = build_fdm_signal(messages, Fs)
% Load configuration
config = system_config();
% Determine number of stations
num_signals = length(messages);
% Define carrier parameters
base_carrier = config.Fc0; % Carrier frequency of first station
deltaF       = config.deltaF;    % Frequency spacing between stations
% Determine lengths of all message signals
lengths = zeros(num_signals,1);
for k = 1:num_signals
 lengths(k) = length(messages{k});
end
% Use the shortest signal length to ensure equal size
N = min(lengths);
% Initialize multiplexed FDM signal
fdm_signal = zeros(N,1);
% Loop through all stations
for n = 1:num_signals
 % Compute carrier frequency for the current station
 Fc = base_carrier + (n-1)*deltaF;
 % Truncate message to the common signal length
 signal = messages{n}(1:N);
```

```
 % Perform DSB-SC modulation
 modulated = dsb_sc_modulate(signal, Fc, Fs);
 % Add modulated signal to the multiplexed FDM signal
 fdm_signal = fdm_signal + modulated;
end
end
```

6. Receiver Design

6.1. RF Stage

The RF stage is implemented as a **band-pass filter** centered at the desired carrier frequency. Its main purpose is to select the target station and reject **image** frequencies and adjacent channels, which is an essential requirement in a **superheterodyne** receiver.

By limiting the spectrum around the selected carrier before the mixing process, the RF filter prevents unwanted signals from being translated to the intermediate frequency during the mixing stage.

The filter bandwidth is chosen to pass the **double-sideband** spectrum of the modulated signal while attenuating other stations in the FDM signal.

The RF stage can be implemented in MATLAB is shown in the following function:

```
function rf_output = rf_stage_filter(signal, Fs, Fc)
% Load system configuration parameters
config = system_config(
% Required RF bandwidth (DSB signal occupies twice the message BW)
BW = 2*config.BW;
% Lower and upper band edges of the RF filter
f1 = Fc - BW/2;
f2 = Fc + BW/2;
% FIR filter length
N = 200;
% Symmetric time index for impulse response
n = -N/2:N/2;
% Convert band edges to digital radian frequencies
wc1 = 2*pi*f1/Fs;
wc2 = 2*pi*f2/Fs;
% Ideal band-pass filter impulse response
h = (sin(wc2*n) - sin(wc1*n)) ./ (pi*n);
% Correct the center sample to avoid division by zero
h(N/2 + 1) = (wc2 - wc1)/pi;
% Apply Hamming window to obtain a practical FIR filter
k = 0:N;
w = 0.54 - 0.46*cos(2*pi*k/N);
h = h .* w;
% Filter the RF signal
rf_output = conv(signal, h, 'same');
end
```

The filter order was chosen as **200** to provide sufficient **selectivity** between adjacent channels while maintaining reasonable **computational complexity**. A Hamming window was applied to the ideal impulse response to reduce **Gibbs oscillations** and obtain a practical FIR filter with good side-lobe suppression

6.2. Mixer Stage

The mixer stage performs frequency translation of the selected RF signal to the intermediate frequency (IF). This is achieved by multiplying the RF signal with a local oscillator (LO) signal whose frequency is chosen as $f_{lo} = f_c + f_{if}$.

The mixing process generates two frequency components corresponding to the sum and difference frequencies. The desired component is the difference frequency, which shifts the selected station to the intermediate frequency **(15 kHz)**. The unwanted components are later removed by the **IF band-pass filter**.

The mixer stage can be implemented in MATLAB is shown in the following function:

```
function if_signal = mixer_stage(rf_signal, Fs, f_LO)
 % Determine number of samples
 N = length(rf_signal);
 % Generate time vector based on sampling frequency
 t = (0:N-1)' / Fs;
 % Generate local oscillator signal
 lo = cos(2*pi*f_LO*t);
 % Perform mixing by multiplying RF signal with LO
 % This shifts the signal spectrum to new frequencies
 if_signal = rf_signal .* lo;
end
```

6.3. IF Stage

The IF stage is implemented as a **band-pass filter** centered at the intermediate frequency (**IF**). After the mixing process, the desired station is translated to the IF frequency while other unwanted components appear at different frequencies.

The purpose of the IF filter is to select the signal located at the intermediate frequency and suppress the remaining mixer products. Similar to the RF stage, the filter bandwidth is chosen to pass the **double-sideband** spectrum of the signal while attenuating out-of-band components. This stage improves selectivity before the signal is demodulated in the baseband stage

The IF stage can be implemented in MATLAB is shown in the following function:

```
function if_output = if_stage_filter(signal, Fs)
 % Load system configuration parameters
 config = system_config();
 % Intermediate frequency center
 F_IF = config.IF;
 % Required bandwidth (DSB signal)
 BW = 2*config.BW;
 % Lower and upper band edges
 f1 = F_IF - BW/2;
 f2 = F_IF + BW/2;
 % FIR filter order
 N = 200;
 % Symmetric index for impulse response
 n = -N/2:N/2;
 % Convert band edges to digital radian frequencies
 wc1 = 2*pi*f1/Fs;
```

```
 wc2 = 2*pi*f2/Fs;
 % Ideal band-pass filter impulse response
 h = (sin(wc2*n) - sin(wc1*n)) ./ (pi*n);
 % Fix center sample (avoid division by zero)
 h(N/2+1) = (wc2 - wc1)/pi;
 % Apply Hamming window for practical FIR filter
 k = 0:N;
 w = 0.54 - 0.46*cos(2*pi*k/N);
 h = h .* w;
 % Filter the IF signal
 if_output = conv(signal, h, 'same');
end
```

6.4. Baseband Mixer

After the **IF** filtering stage, the signal is translated from the intermediate frequency (**15 kHz**) back to **baseband**.

This is achieved by multiplying the IF signal with a **local oscillator** operating at the same IF frequency. The mixing process generates sum and difference frequency components, where the difference component shifts the desired signal spectrum to baseband. The resulting signal still contains high-frequency components produced during mixing, which are later removed using a **low-pass filter** in the baseband stage.

The baseband mixer can be implemented in MATLAB is shown in the following function:

```
function baseband_signal = baseband_mixer(if_signal, Fs)
 % Load system configuration parameters
 config = system_config();
 % Intermediate frequency used in the receiver
 f_IF = config.IF;
 % Determine signal length
 N = length(if_signal);
 % Generate time axis
 t = (0:N-1)'/Fs;
 % Generate local oscillator at IF frequency
 lo = cos(2*pi*f_IF*t);
 % Mix IF signal with local oscillator to shift it to baseband
 baseband_signal = if_signal .* lo;
end
```

6.5. Baseband LPF

After the baseband mixing stage, the signal contains the desired audio component around DC as well as higher-frequency components produced during the mixing process.

A **low-pass filter** is therefore applied to recover the original audio signal. The filter passes frequencies within the **audio bandwidth** while attenuating the higher-frequency components generated during mixing.

The filter is implemented as a finite impulse response **(FIR)** filter obtained from an ideal low-pass response that is windowed using a **Hamming window** to produce a **practical** implementation.

The baseband LPF can be implemented in MATLAB is shown in the following function:

```
function audio_out = baseband_lpf(signal, Fs)
 % Load system configuration parameters
 config = system_config();
 % Audio bandwidth (baseband signal occupies BW)
 BW = config.BW;
 % FIR filter order
 N = 200;
 % Symmetric time index for impulse response
 n = -N/2:N/2;
 % Convert cutoff frequency to digital radian frequency
 wc = 2*pi*BW/Fs;
 % Ideal low-pass filter impulse response (sinc)
 h = sin(wc*n)./(pi*n);
 % Correct center sample to avoid division by zero
 h(N/2+1) = wc/pi;
 % Generate Hamming window
 k = 0:N;
 w = 0.54 - 0.46*cos(2*pi*k/N);
 % Apply window to obtain practical FIR filter
 h = h .* w;
 % Apply filter to recover baseband audio
 audio_out = conv(signal,h,'same');
end
```

6.6. Baseband Decimation

After the baseband filtering stage, the signal is still sampled at the **high sampling frequency** used during interpolation and RF processing. To return the signal to its original audio sampling rate, a **decimation** step is applied.

This process reduces the sampling rate by a factor equal to the interpolation factor used earlier in the transmitter. By keeping every $\boldsymbol{L_{th}}$ sample, the sampling frequency is reduced back to the original audio rate while preserving the recovered baseband signal.

The decimation step can be implemented in MATLAB is shown in the following function:

```
function signals_out = decimate_signal(signals_in, L)
% Extract signal and current sampling frequency
signal = signals_in.signal;
Fs     = signals_in.Fs;
% Downsample the signal by factor L
signal_decimated = signal(1:L:end);
% Update the sampling frequency
Fs_new = Fs / L;
% Store outputs in a structure
signals_out.signal = signal_decimated;
signals_out.Fs = Fs_new;
end
```

7. Super Heterodyne Receiver

In this stage, the full superheterodyne receiver architecture is implemented by connecting the previously developed RF, mixing, IF filtering, baseband processing, and decimation blocks.

```
function audio = receiver_system(fdm_signal, Fs, config)
% Compute selected carrier frequency
Fc = config.Fc0 + (config.station-1)*config.deltaF;
% Compute local oscillator frequency
f_LO = Fc + config.IF;
% RF Stage
% Select desired station using band-pass filter
rf_signal = rf_stage_filter(fdm_signal, Fs, Fc);
% Mixer: convert RF ? IF
mixed_signal = mixer_stage(rf_signal, Fs, f_LO);
% IF Stage Filter
if_signal = if_stage_filter(mixed_signal, Fs);
% Mixer: convert IF ? baseband
baseband_signal = baseband_mixer(if_signal, Fs);
% Baseband Low-Pass Filter
% Recover audio band (0-5 kHz)
audio_high_rate = baseband_lpf(baseband_signal, Fs);
% Decimation
% Reduce sampling rate back to audio rate
temp.signal = audio_high_rate;
temp.Fs = Fs;
signals_out = decimate_signal(temp, config.L);
audio = signals_out.signal;
end
```

8. Verification and Testing

8.1. Verification Plan

Label	Design Requirement Describtion	Simulation Method	Functionality Check
TEST_1	Signal is loaded and can be plotted	Plot spectrum of loaded signal	A checker in verification tests to ensure successful signal loading
TEST_2	Interpolation increase signal FS by factor L	Interpolation estimation and plot comparison	A checker in verification tests to ensure proper interpolation and following Nyquist rate
TEST_3	DSB modulation shift the baseband signal to higher frequncy	plot spectrum of each station and their FDM	A checker in verification tests to ensure proper modulation and FDM
TEST_4	RF stage apply BPF to select the required station and reject imaging	plot RF spectrum after RF stage	A checker in verification tests to ensure proper RF filtering and image rejection
TEST_5	RF_to_IF mixer shift the signal to the IF and higher side frequency	plot RF signal after mixing	A checker in verification tests to ensure proper mixing and accurate LO
TEST_6	IF stage apply BPF to select the required station to downconverted to baseband	plot IF signal after filtering	A checker in verification tests to ensure proper IF filtering and station selection
TEST_7	IF_to_BB mixer shift the signal to the baseband frequency to be restored	plot output signal after mixing	A checker in verification tests to ensure proper mixing and accurate LO
TEST_8	Baseband LPF reject higher frequency components and restore the signal spectrum	plot the spectrum of resulted BB signal	A checker in verification tests to ensure proper BB filtering and signal restoring
TEST_9	Decimation lower FS to restore original sampling state	Applying decimation and estimate FS for each signal	A checker in verification tests to ensure proper restoring of sample frequency
TEST_10	Signal should me recovered as similar as possible to the transmitted signal	plot original loaded signal VS the received one after decimation	A checker in verification tests to ensure proper signal recovery
TEST_11	all tests should run successfully when applying them in sequencial process	use run_tests file directly	A checker in verification tests to ensure proper sequencial operation
TEST_12	Removing RF filter results in existence of the corresponding signal images	plot spectrum of IF and mixer and BB LPF and listening to received audio	A checker in verification tests to see the effect of imaging after removal of RF filter
TEST_13	The offset in local oscillator frequency results in signal distortion	introduce LO offset and listen to the received audio	A checker in verification tests to see the effect of LO frequency offset

Figure 4: Verification Plan For Super Heterodyne Receiver

8.2. Verification Tests

8.2.1. Test Signal Loading

The idea of the test is to check the signal is loaded successfully to MATLAB and plotting its baseband spectrum for visualization

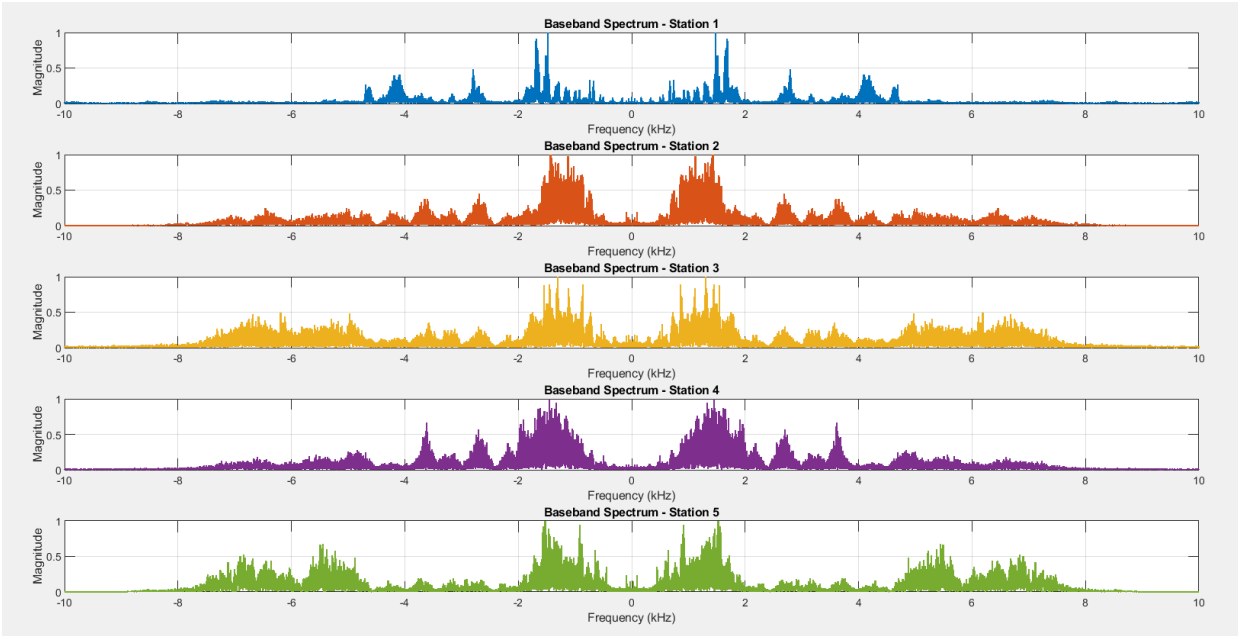


Figure 5: FD Spectrum of Original Loaded Audio Signals

From the plot it's obvious that signal power attenuates in around **6 KHz** which is the previously estimated bandwidth

8.2.2. Test Interpolation

The aim of the test is to apply interpolation by our analyzed factor $\mathbf{L = 60}$ and to test if the produced signal has sampling frequency **L multiple** of original sampling frequency and visualize both cases

```
>> test_interpolation
TEST: INTERPOLATION
Original Fs: 44100
Number of signals : 5
New Fs: 2646000
Interpolation test PASSED
```

Figure 6: New Sampling Frequency Estimation in MATLAB

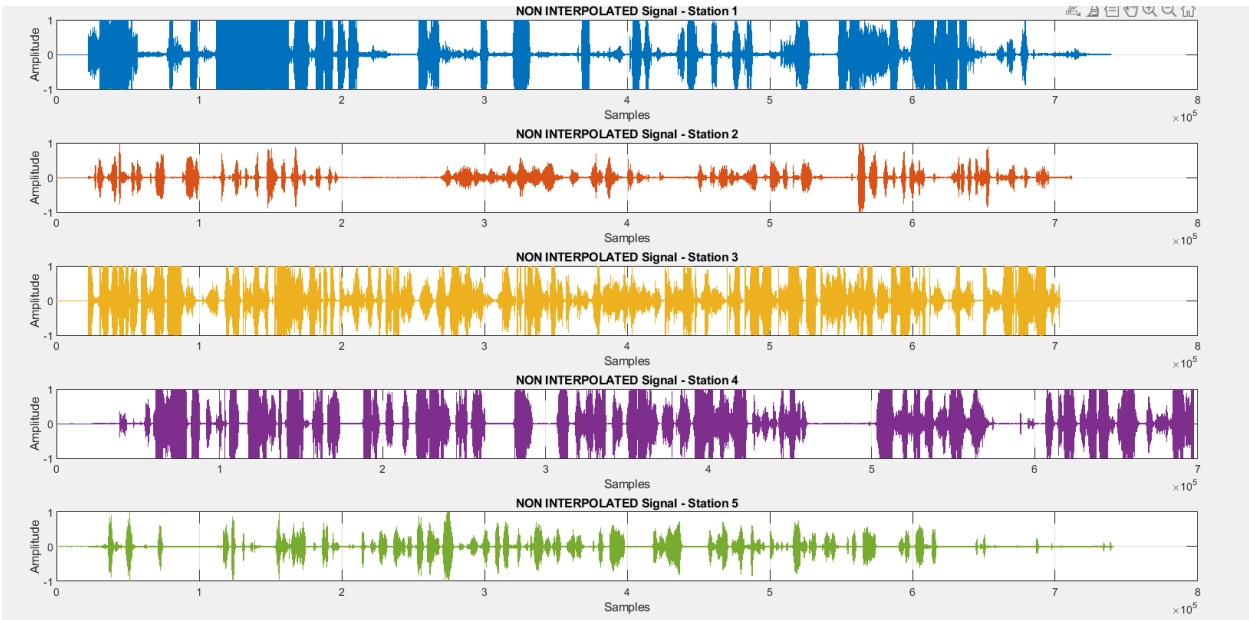


Figure 7: Sample Plot of Original Loaded Signal Before Interpolation

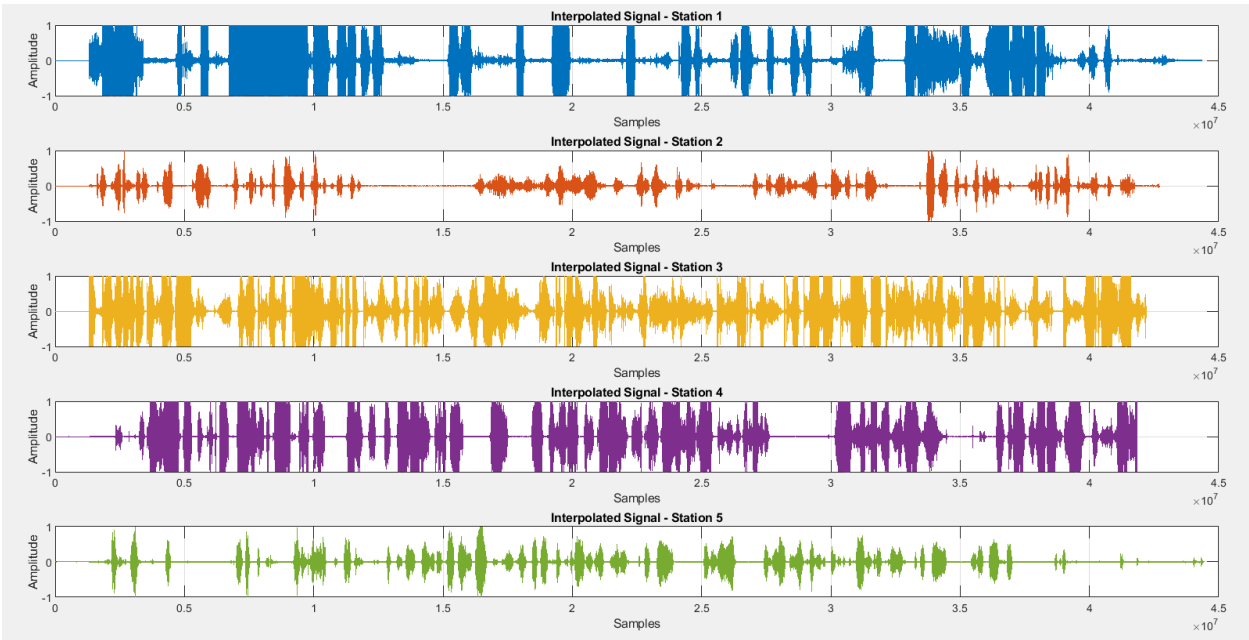


Figure 8: Sample Plot of The Interpolated Signal

8.2.3. Test Modulation

The aim of the test is to apply **FFT** and visualize the **baseband spectrum** of the signal before modulation and apply the **modulation** function and observe how the spectrum is shifted to higher frequency

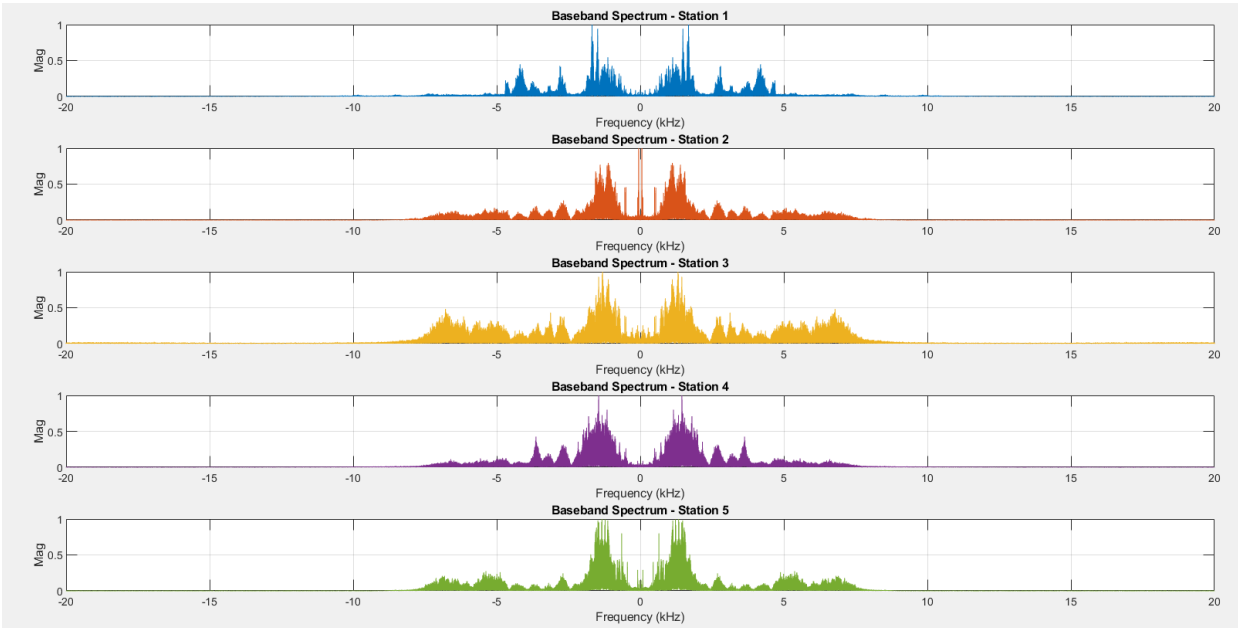


Figure 9: Baseband Spectrum of Transmitted Signal

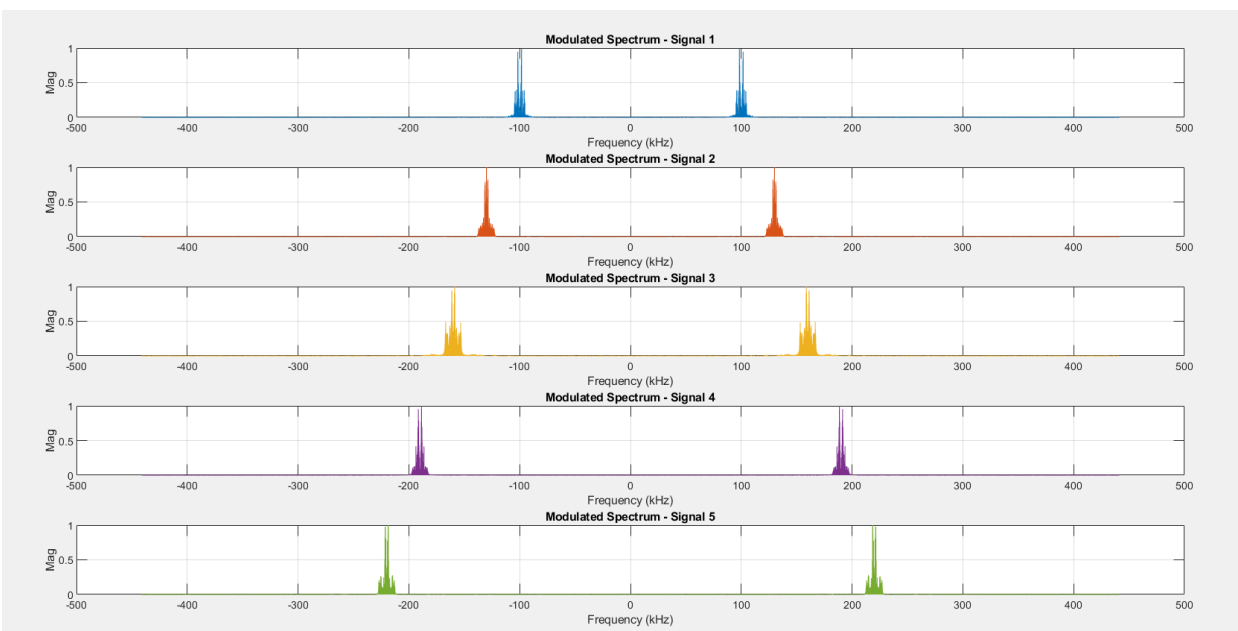


Figure 10: RF Spectrum of Each Individual Signal After Modulation

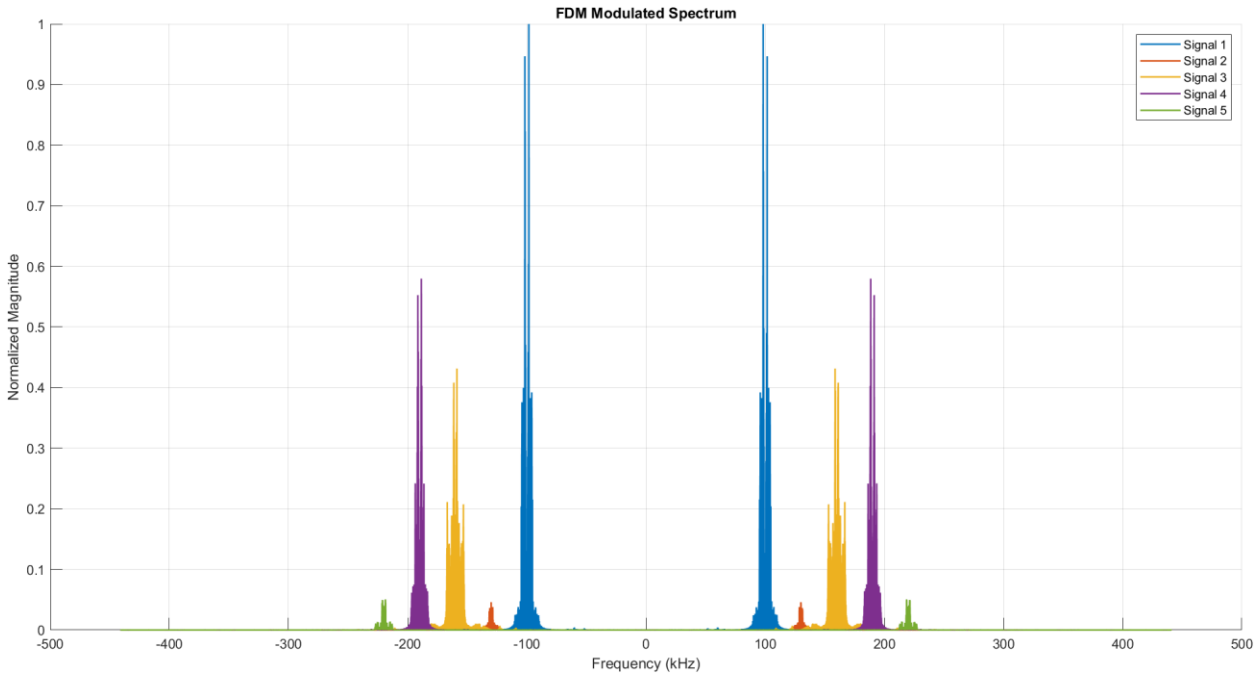


Figure 11: FDM of All Transmitted Signals

8.2.4. Test RF Stage

The aim of this test is to that RF bandpass filter works properly and can filter the required station just by calling its function any apply it on the FDM with the tunable carrier frequency

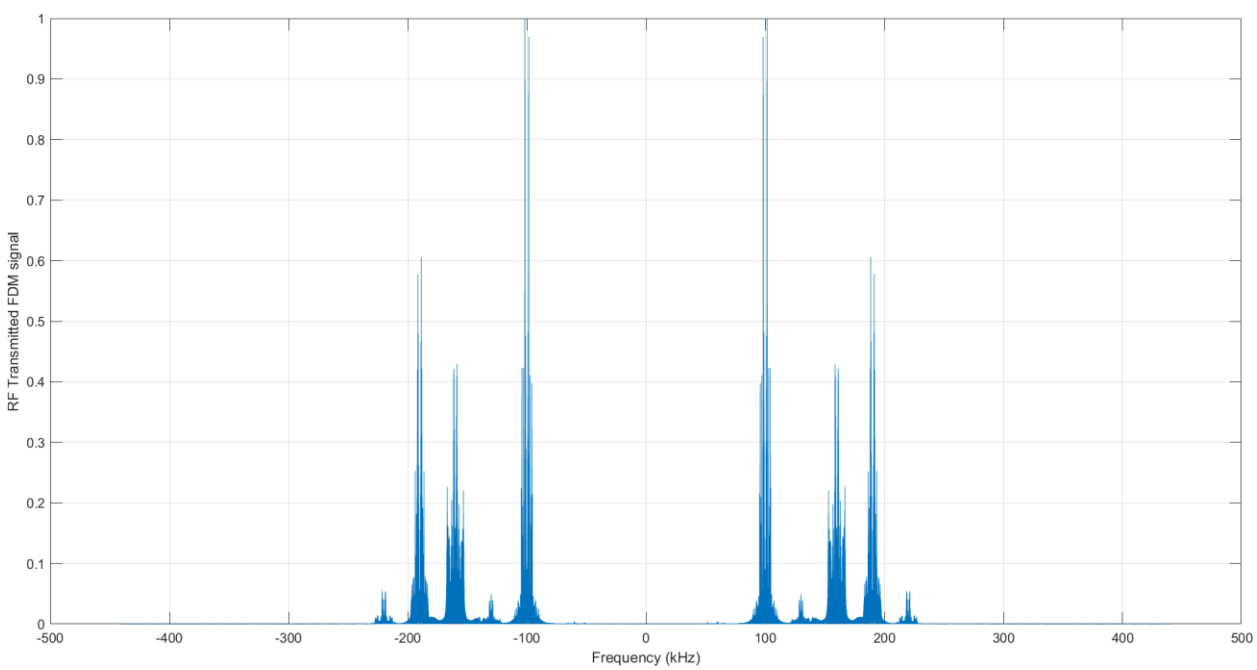


Figure 12: Received RF Spectrum at The Receiver

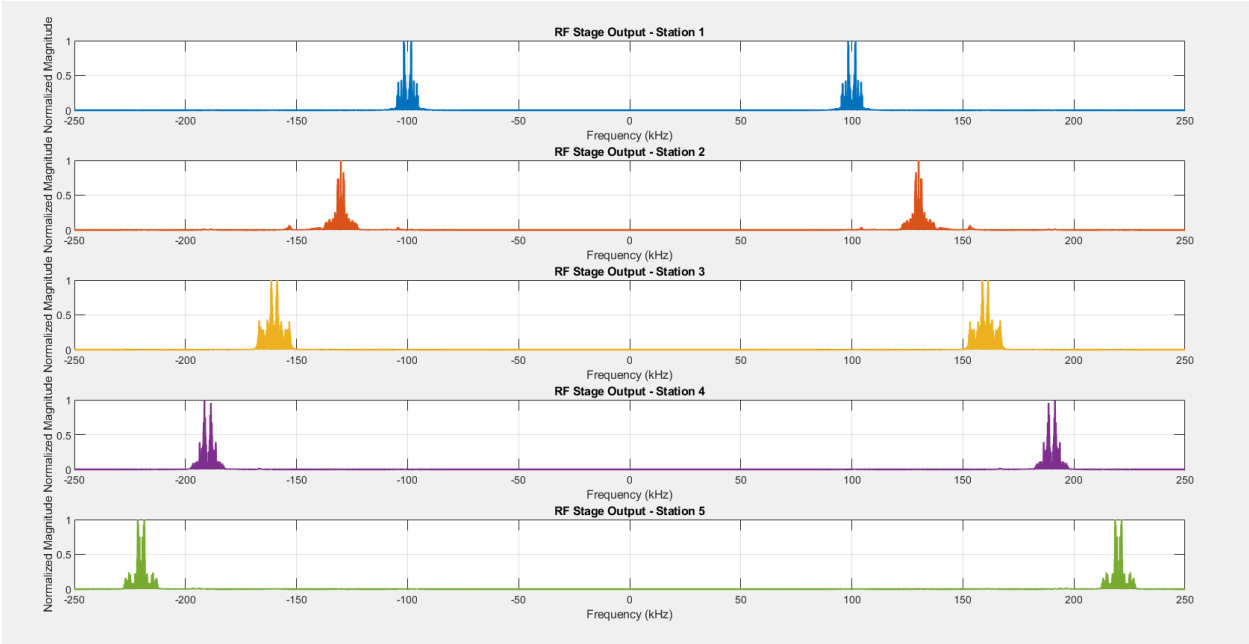


Figure 13: Selected RF Signal after Bandpass Filter

8.2.5. Test Mixer Stage

The aim of this test is to ensure that RF mixer works properly and shift RF spectrum left to the IF stage and right in high frequences

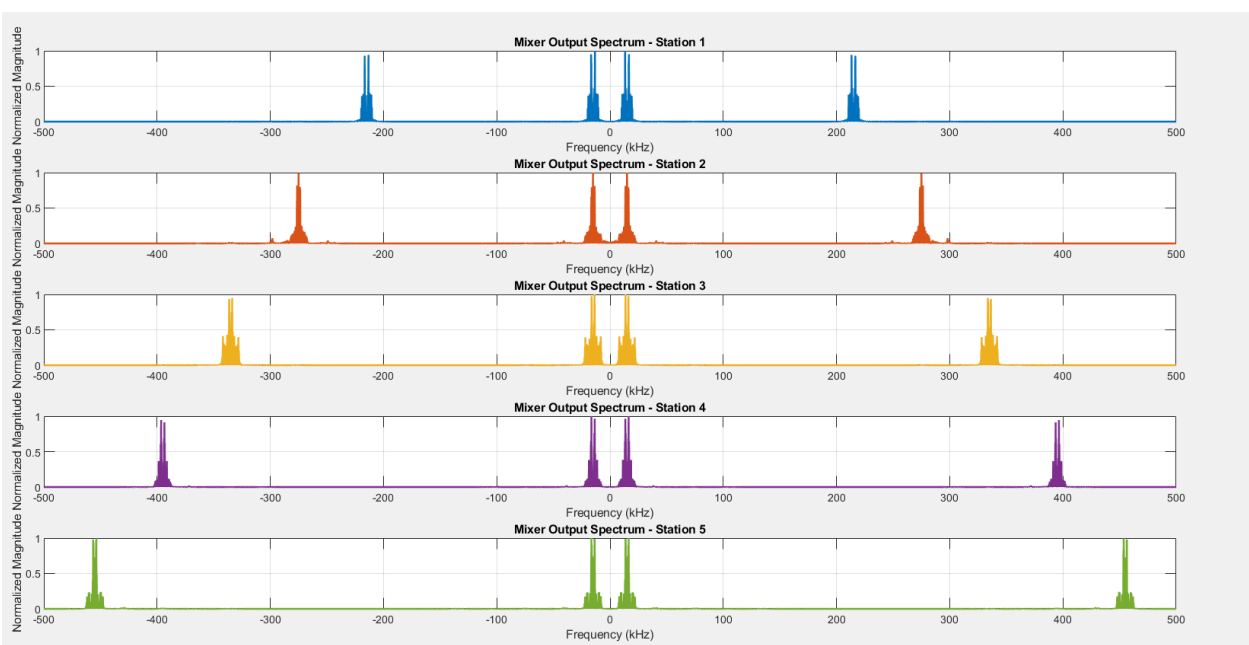


Figure 14: The Produced Spectrum After Mixing to IF

8.2.6. Test IF Stage

This test aims to ensure proper operation of bandpass filter at the intermediate frequency

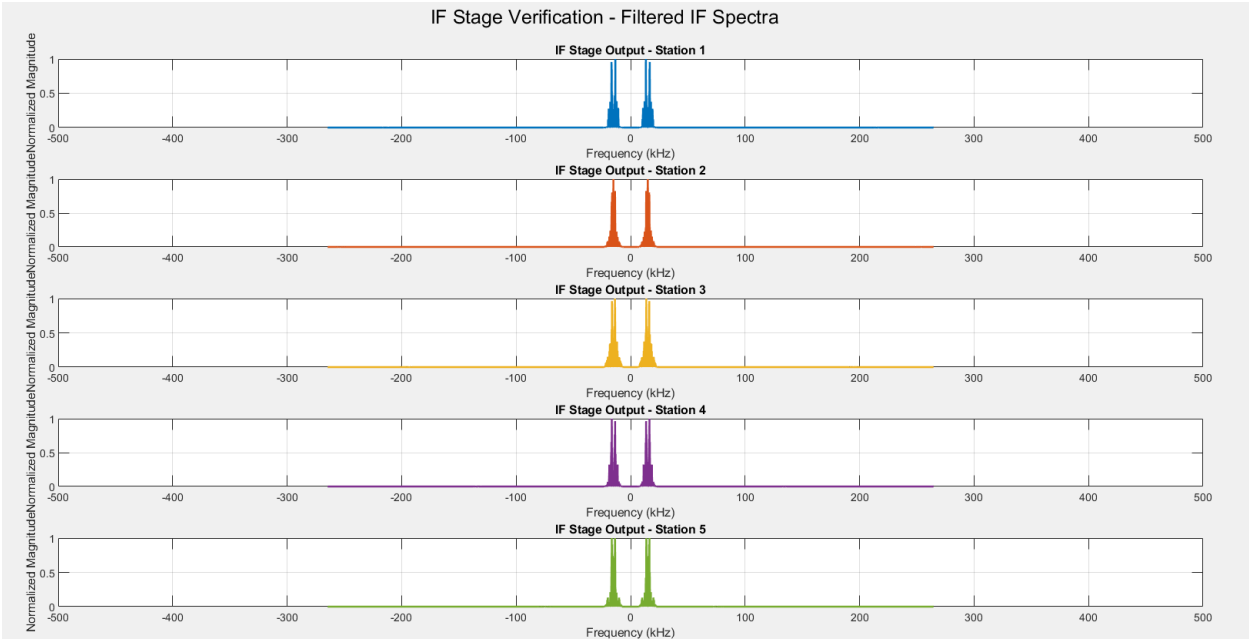


Figure 15: The Filtered Spectrum After IF Filter

8.2.7. Test Baseband Mixer Stage

This test ensure that the intermediate frequency mixer works properly and shift the signal to the baseband successfully

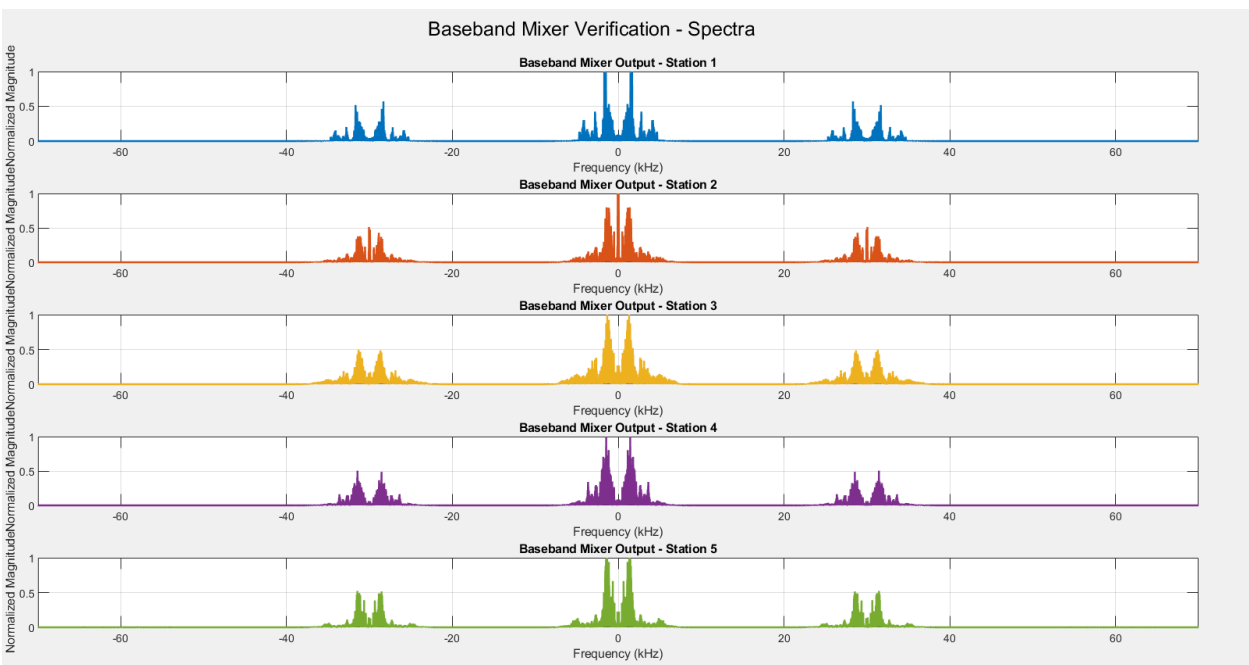


Figure 16: The Produced Spectrum After Mixing to Baseband

8.2.8. Test Baseband LPF

This test ensures proper operation of LPF in filtering baseband spectrum and geting the final received signal

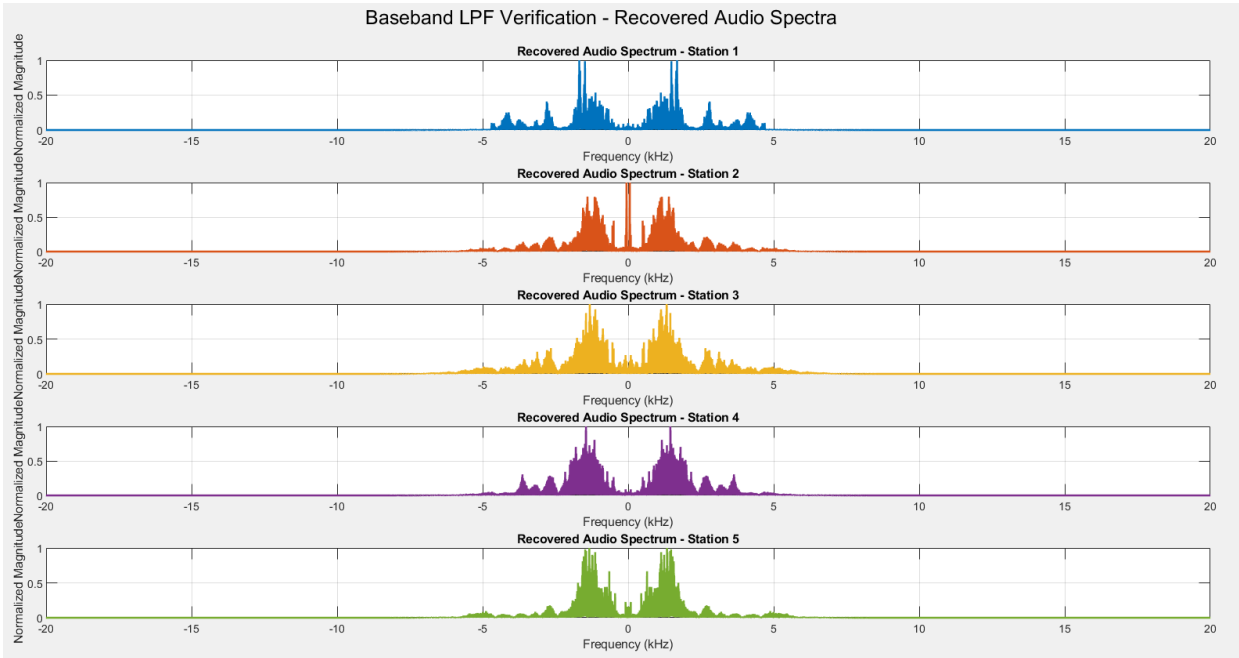


Figure 17: Baseband Spectrum of The Received Audio Signal

8.2.9. Test The Decimation Step

This test aims to ensure successful down sampling and restoring the original sampling rate of the audio

```
>> test_decimation
=== TEST: DECIMATION ===
Number of stations loaded: 5
FDM signal generated
Testing decimation for station 1
 Recovered sampling frequency: 44100
Testing decimation for station 2
 Recovered sampling frequency: 44100
Testing decimation for station 3
 Recovered sampling frequency: 44100
Testing decimation for station 4
 Recovered sampling frequency: 44100
Testing decimation for station 5
 Recovered sampling frequency: 44100
fx >>
```

Figure 18: Estimated Sampling Frequency After Interpolation

8.2.10. Test Signal Recovery

This test compares the decimated received signal with the original non-interpolated message to evaluate the accuracy of the signal reconstruction and verify proper receiver functionality.

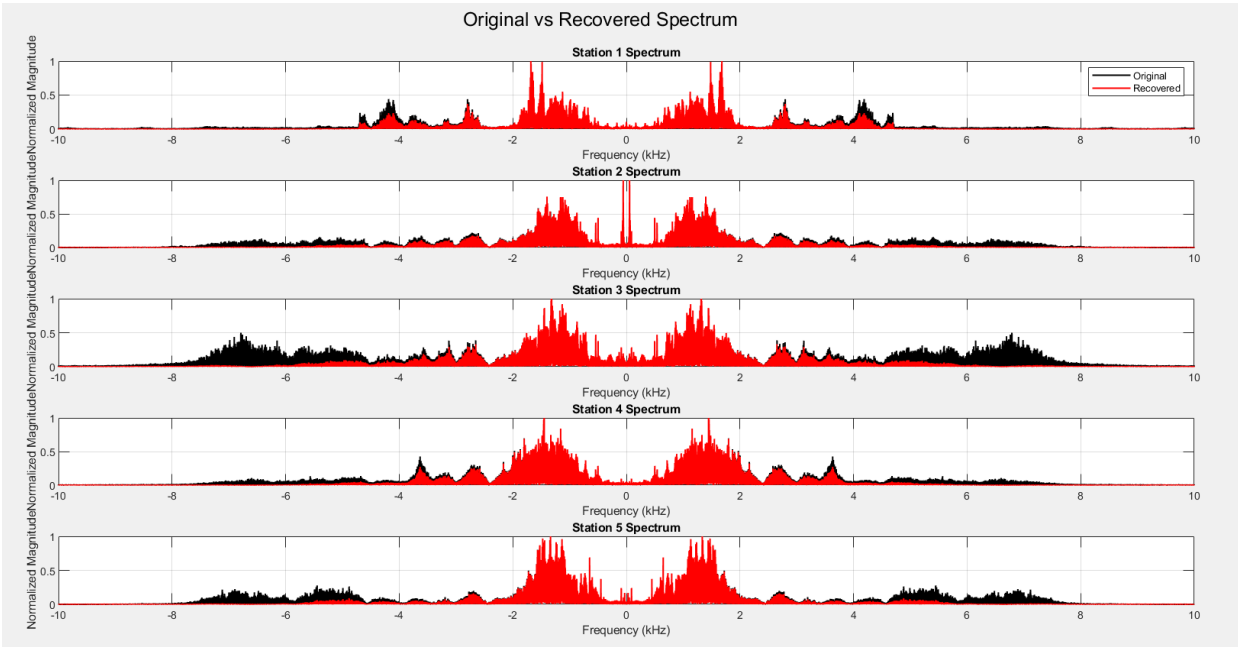


Figure 19: The Spectrum of The Received Audio Compared to The Transmitted One

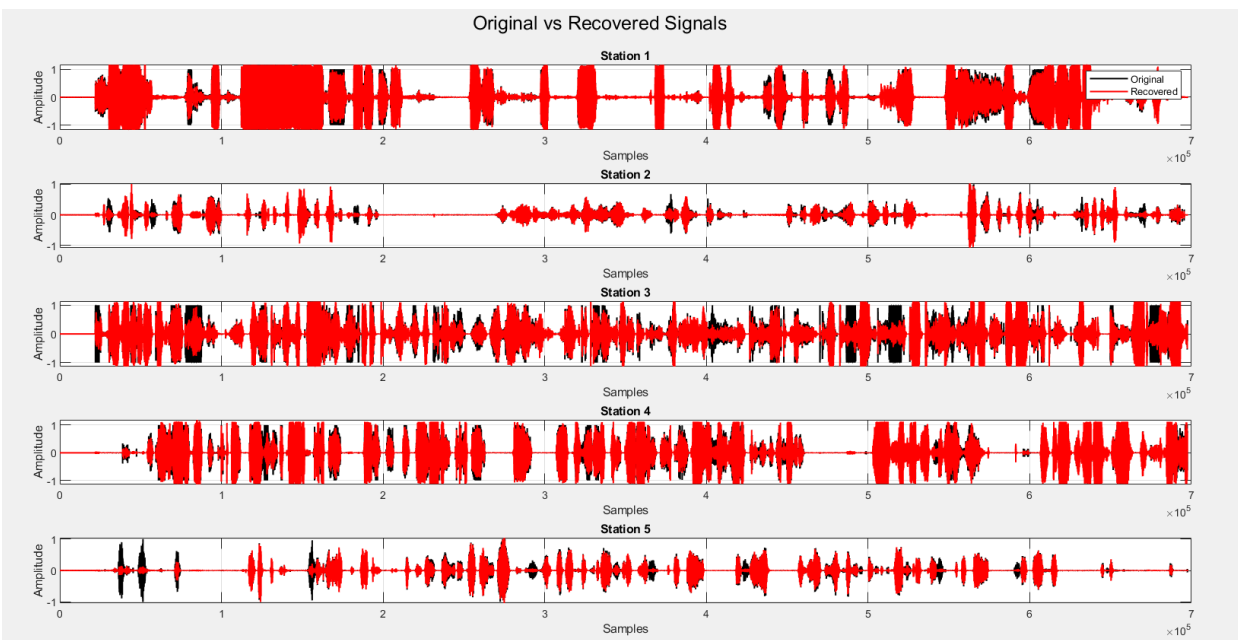


Figure 20: The Sample Plot of The Received Audio Compared to The Transmitted One

Both spectra are normalized to a common scale to compensate for attenuation in the received signal and allow a direct visual comparison.

8.3. Automated Test File

The aim of this file is to run all receiver processing blocks sequentially to ensure that the signal propagates correctly through the entire receiver architecture.

```
clc
clear
close all
addpath(genpath(pwd))
disp("========================================")
disp(" RUNNING SUPERHETERODYNE TEST SUITE ")
disp("========================================")
tests = {
    "test_interpolation"
    "test_modulation"
    "test_rf_stage"
    "test_mixer_stage"
    "test_if_stage"
    "test_baseband_mixer"
    "test_baseband_lpf"
    "test_decimation"
    "test_audio_recovery"
    "test_signal_quality"
    "test_station_selection"
};
for k = 1:length(tests)
    test_name = tests{k};
    disp("----------------------------------------")
    disp("Running " + test_name)
    disp("----------------------------------------")
    close all
    run("tests/" + test_name + ".m")
end
disp("========================================")
disp(" ALL TESTS COMPLETED SUCCESSFULLY ")
disp("========================================")
```

9. Main System

This script executes the **complete superheterodyne receiver** system to recover a **selected** audio station from the **multiplexed** signal.

Initially, the system **configuration** and input audio signals are **loaded**. The signals are then **interpolated** to increase the sampling rate and combined to form the multiplexed **FDM** transmitter signal.

The receiver system subsequently processes this signal through its internal stages to extract the desired station. The recovered audio signal is then normalized, played for verification, and saved to a file for further analysis and evaluation.

The main code is implemented in MATLAB as the following:

```
clc
clear
close all
disp("==========================================")
disp(" SUPERHETERODYNE RECEIVER SYSTEM ")
disp("==========================================")
%% Load configuration
disp("Loading system configuration...")
config = system_config();
%% Load signals
disp("Loading audio stations...")
signals = load_signals();
%% Interpolation
disp("Interpolating signals (increase sampling rate)...")
signals = interpolate_signal(signals, config.L);
Fs = signals.Fs;
messages = signals.messages;
%% Build FDM signal
disp("Building FDM multiplexed transmitter signal...")
fdm_signal = build_fdm_signal(messages, Fs);
%% Receiver
disp("Running receiver system...")
audio = receiver_system(fdm_signal, Fs, config);
%% Normalize
disp("Normalizing recovered audio...")
audio = audio / max(abs(audio));
%% Audio sampling frequency
Fs_audio = Fs / config.L;
%% Playback
disp("Playing recovered station...")
sound(audio, Fs_audio)
%% Save audio
disp("Saving recovered audio to results folder...")
audiowrite("results/recovered_station.wav", audio, Fs_audio)
disp("==========================================")
disp(" SYSTEM EXECUTION COMPLETED ")
disp("==========================================")
```

- When testing the whole receiver system on all stations (file must be in Word format):

Station1.wav Station2.wav Station3.wav Station4.wav Station5.wav

10. Experiments and Results

10.1. Removing RF Bandpass Filter

When the RF band-pass filter is removed, all multiplexed stations enter the mixer simultaneously. The mixing operation translates each carrier frequency to a different intermediate frequency given by $f_{IF} = | f_n - f_{LO} |$.

As a result, multiple stations appear in the IF spectrum instead of a single desired component at 15 kHz.

This causes interference and prevents the IF filter from isolating the desired station, which leads to degraded signal recovery.

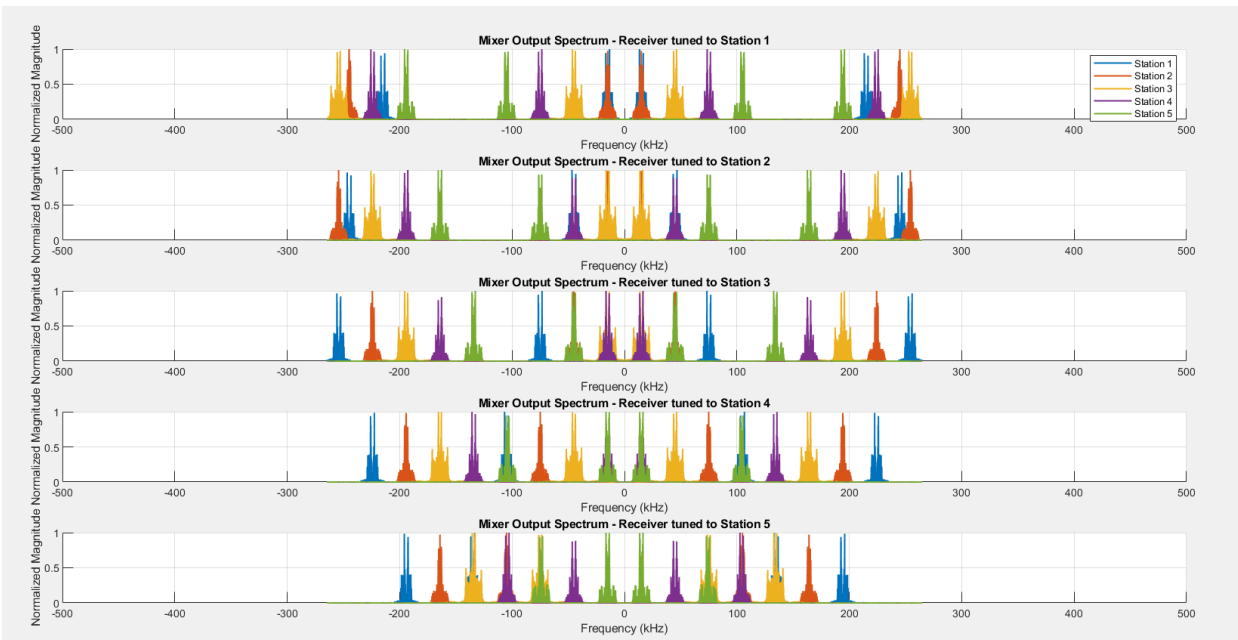


Figure 21: Imaging Occurrence Due to Removing RF Stage

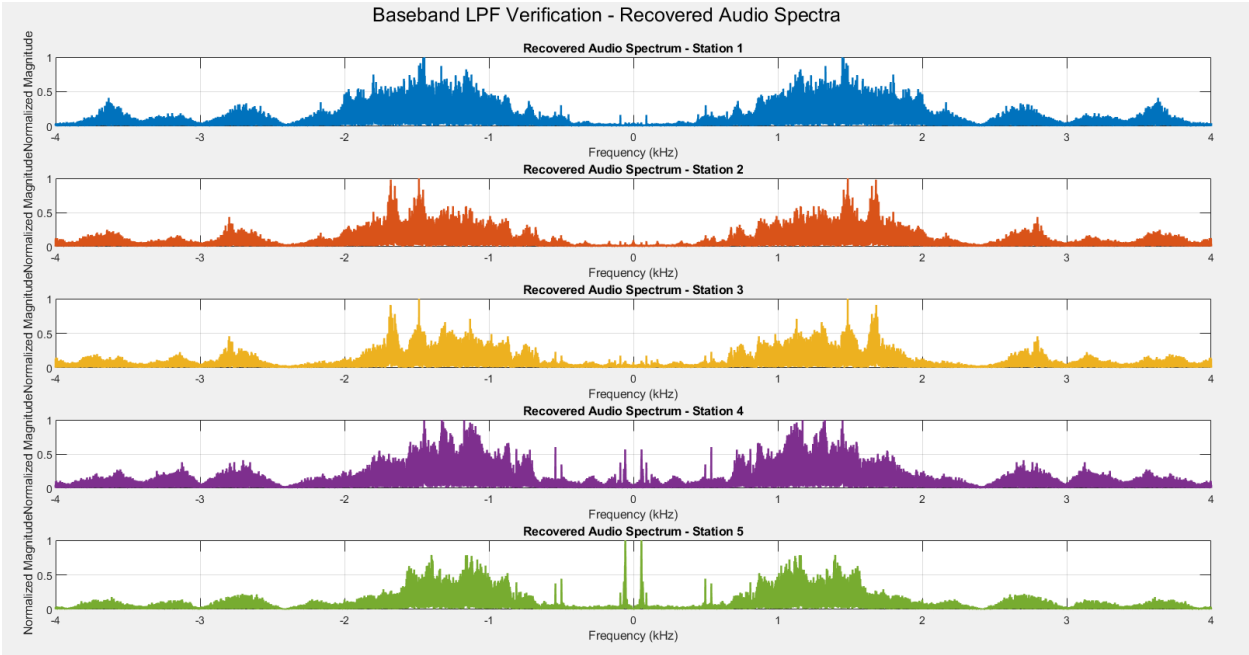


Figure 22: Interference between Signals at BP due to Imaging

- **Audio observation:** when we are trying to select certain station, we observe that another station in played as a plus in the back ground which is considered the image of our selected station

RF_filter_removal_selecting1.wav RF_filter_removal_selecting2.wav RF_filter_removal_selecting3.wav RF_filter_removal_selecting4.wav RF_filter_removal_selecting5.wav

10.2. Receiver Local Oscillator Mismatch

With a small **local oscillator offset** of 0.1 kHz (100 Hz), the recovered audio is only slightly affected. This is because the offset introduces a **small frequency shift** in the down converted signal, resulting in a low-frequency **beating** effect and a subtle pitch variation.

Although this shift is minor, it can still slightly degrade signal quality.

However, with a larger offset of 1 kHz, the degradation becomes much more significant. The dominant effect here is the frequency shift introduced during mixing, which moves the entire signal spectrum away from its intended position.

As a result, the signal becomes **misaligned** with the **IF** filter passband, leading to distortion, attenuation of key frequency components, and a pronounced shift in audio pitch, making the sound clearly unnatural and degraded.

➢ 0. 1 KHz

station1_with_0.1KHz_offset.wav station2_with_0.1KHz_offset.wav station3_with_0.1KHz_offset.wav station4_with_0.1KHz_offset.wav station5_with_0.1KHz_offset.wav

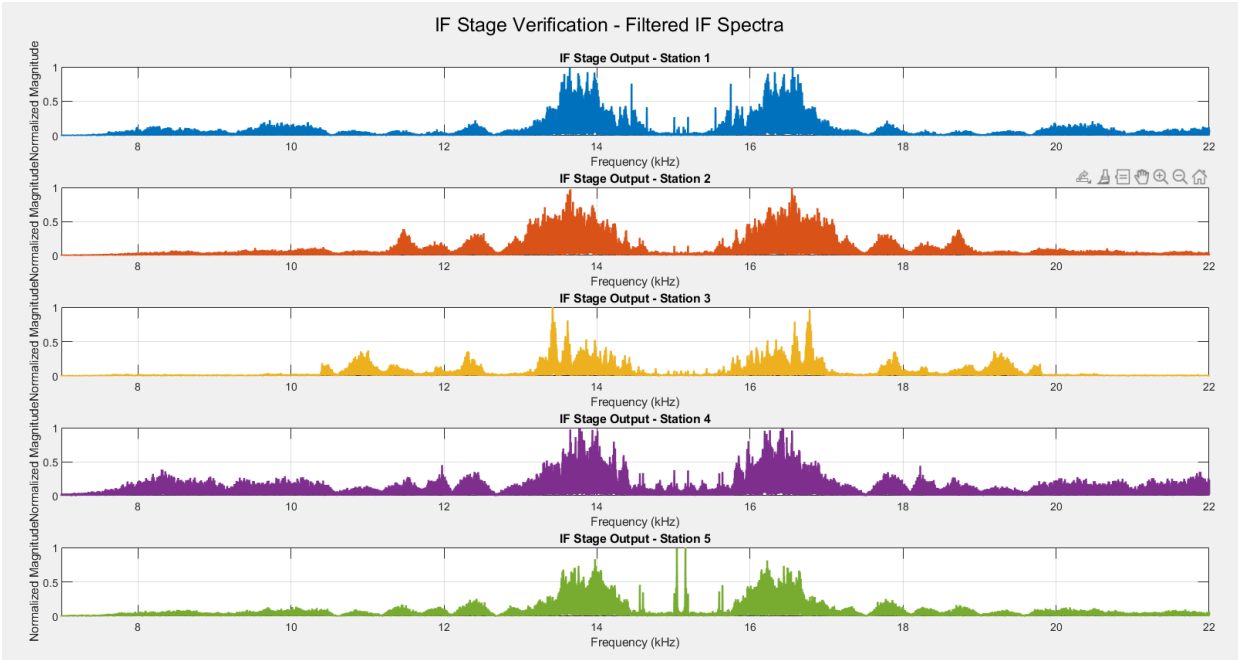


Figure 23: Frequency Shift from IF due to LO Offset 0.1 KHz

Hence the small offset of 0.1 KHz is hard to notice on the spectrum but has it distortive effect on sound

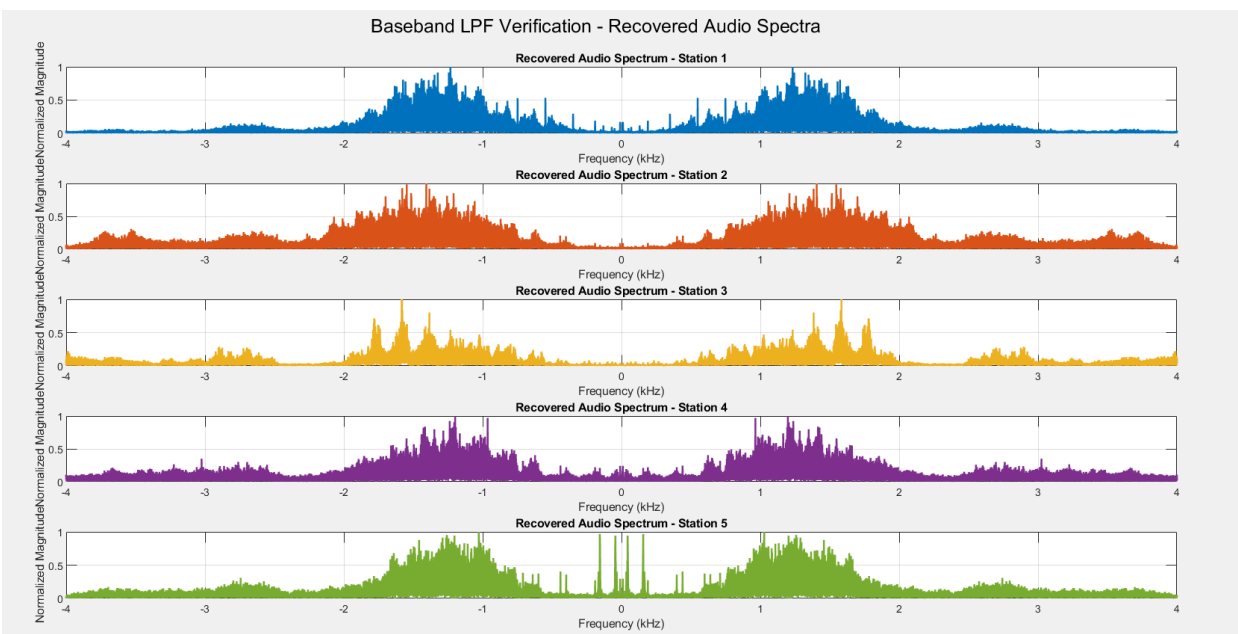


Figure 23: Small Frequency Shift at BP due to LO Offset

➢ 1 KHz

station1_with_1KHz_ofset.wav | station2_with_1KHz_ofset.wav | station3_with_1KHz_ofset.wav | station4_with_1KHz_ofset.wav | station5_with_1KHz_ofset.wav

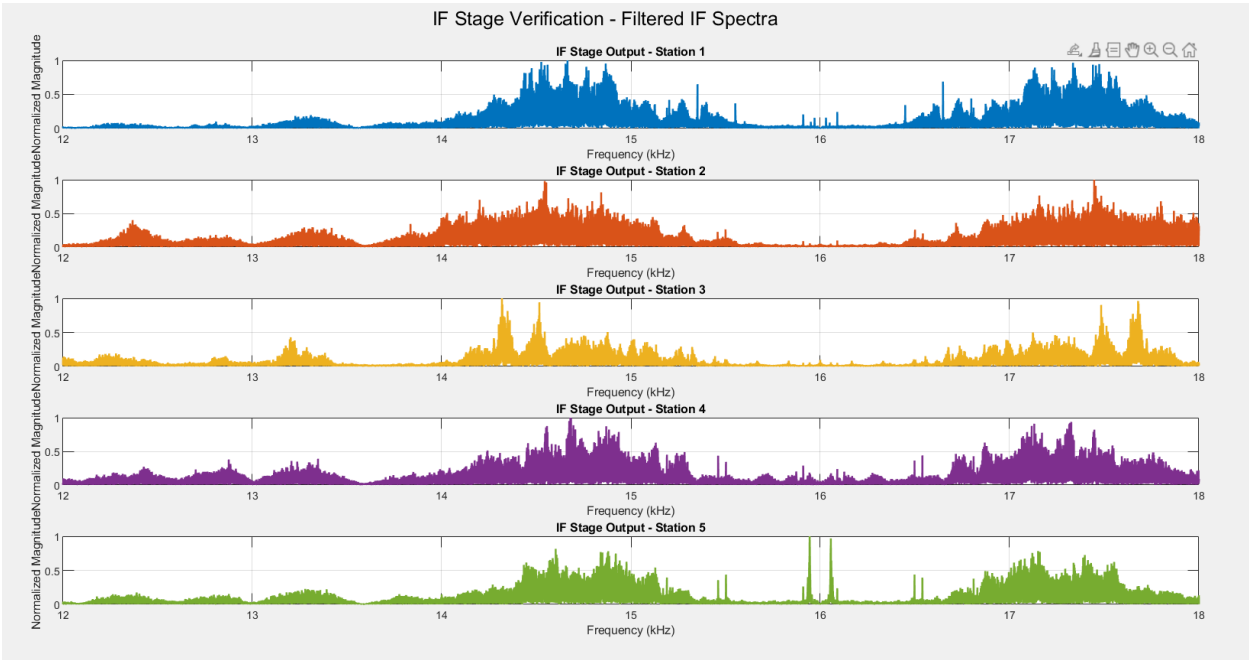


Figure 24:Obvious Frequency Shift at IF Stage Due to 1 KHz LO Offset

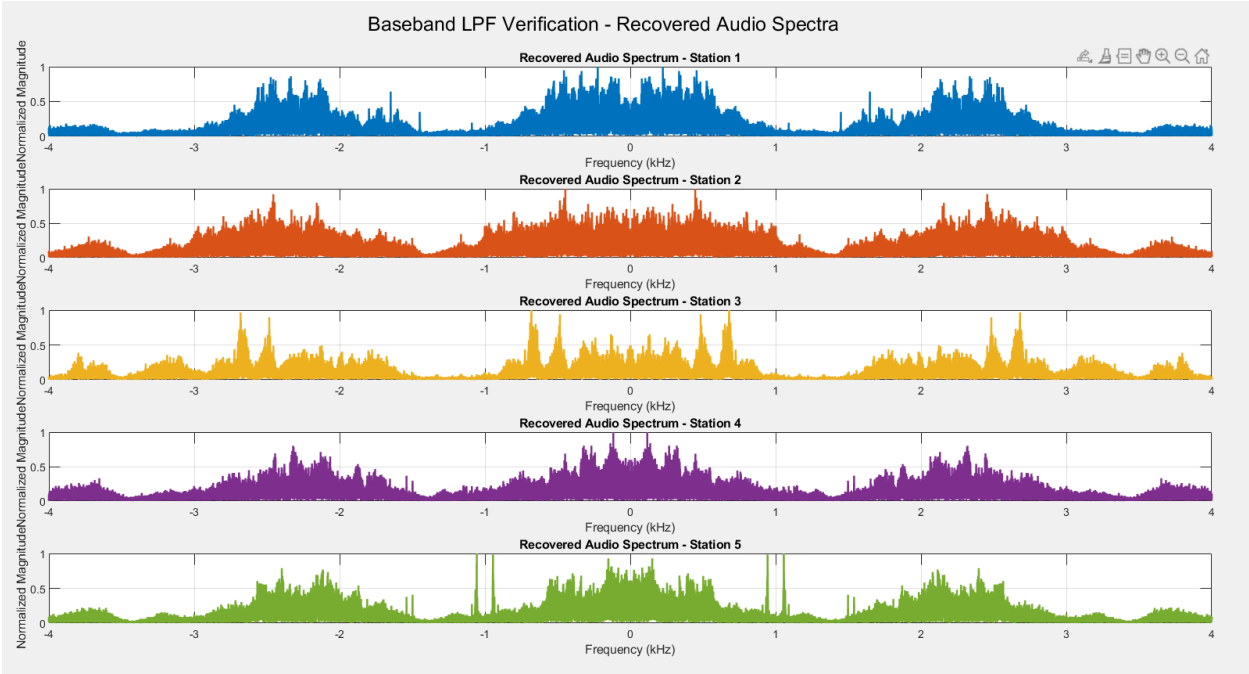


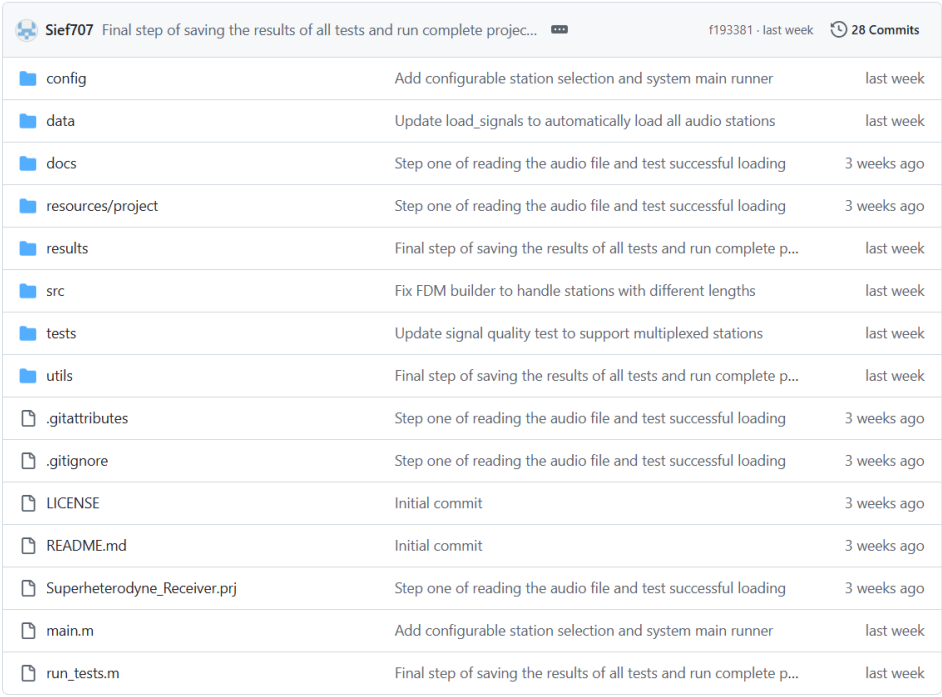
Figure 25: Obvious Frequency Shift at BP Stage Due to 1 KHz LO Offset Which Ended up to Distorted Signal

11. Version Control and Project Organization

Git and **GitHub** were used throughout the entire project to manage both **development** and **testing** in a structured and systematic way. The communication system was built step by step, where each stage (RF processing, IF stage, demodulation, and final audio recovery) was implemented, tested, and **committed** separately.

Version control was essential in this project because of the **strong dependency** between system blocks. Any modification in one stage could affect overall performance, so Git was used to maintain stable versions, track changes, and safely test improvements without losing previous results.

In addition to code development, Git was also used to track testing procedures and generated results, ensuring that performance evaluation and verification steps were well documented and reproducible. This made debugging more efficient and helped validate the system at each stage before **full integration**.



Sief707 Final step of saving the results of all tests and run complete projec... f193381 · last week 28 Commits

config	Add configurable station selection and system main runner	last week
data	Update load_signals to automatically load all audio stations	last week
docs	Step one of reading the audio file and test successful loading	3 weeks ago
resources/project	Step one of reading the audio file and test successful loading	3 weeks ago
results	Final step of saving the results of all tests and run complete p...	last week
src	Fix FDM builder to handle stations with different lengths	last week
tests	Update signal quality test to support multiplexed stations	last week
utils	Final step of saving the results of all tests and run complete p...	last week
.gitattributes	Step one of reading the audio file and test successful loading	3 weeks ago
.gitignore	Step one of reading the audio file and test successful loading	3 weeks ago
LICENSE	Initial commit	3 weeks ago
README.md	Initial commit	3 weeks ago
Superheterodyne_Receiver.prj	Step one of reading the audio file and test successful loading	3 weeks ago
main.m	Add configurable station selection and system main runner	last week
run_tests.m	Final step of saving the results of all tests and run complete p...	last week

Figure 27: Git Interface of The Project Repo

The repository link is included to provide the complete system coding and successive verification tests codes used in the project: https://github.com/Sief707/Super-heterodyne_Receiver

12. Conclusion

In this project, a complete **superheterodyne** receiver system was successfully designed and implemented using **MATLAB**. The system demonstrated the full communication chain, starting from audio signal processing, modulation, and frequency division multiplexing, up to signal reception and recovery.

A **modular** and well-structured **software architecture** was used, allowing each stage of the system to be developed, tested, and verified independently before full integration.

The use of **interpolation** enabled proper RF simulation without aliasing, while the receiver stages (RF filter, mixer, IF filter, and baseband processing) effectively extracted the desired station from the multiplexed signal.

The results showed that the receiver is capable of recovering the transmitted audio with acceptable quality, and the verification tests confirmed correct operation of each subsystem. Additionally, experiments such as removing the RF filter and introducing local oscillator mismatch highlighted important practical challenges like image interference and frequency offset effects.

Finally, the use of **version control tools** ensured organized development, easier debugging, and reliable system validation throughout the project. Overall, this work demonstrates a clear understanding of superheterodyne receiver principles and their practical implementation in a digital simulation environment.